

COMPLETE AI INTERVIEW NOTES

Built around BOLVEDA & SOCHGRID — Campus Intern Drive Edition
Everything you need to explain your projects and ace LLM interviews

How to Use These Notes

These notes cover exactly the theory you need — no more, no less. Every section maps directly to something you built in BOLVEDA or SOCHGRID. Read a section, then answer the interview questions out loud without looking. The cheat sheet at the end is for the morning of your interview.

Section	What It Covers	Your Project Link
1. LLMs	What LLMs are, how they work, base vs tuned	Foundation of everything you built
2. Tokenization	Tokens, BPE, cost, context limits	API cost calculation, chunk sizing
3. Transformer	Self-attention, Q/K/V, multi-head, residuals	Why GPT-4o can follow long prompts
4. Training & RLHF	Pre-training, SFT, reward model, PPO	Why GPT-4o follows your system prompt
5. Embeddings	Dense vectors, cosine similarity, models	BOLVEDA Layer 3 — core of retrieval
6. RAG	Full 6-stage pipeline, chunking, anti-hallucination	BOLVEDA — entire architecture
7. Vector Databases	ChromaDB, HNSW, ANN, why not SQL	BOLVEDA Layer 4
8. Prompt Engineering	Zero-shot, few-shot, CoT, system prompts	BOLVEDA anti-hallucination prompt
9. Agents & ReAct	Tool use, Thought/Action/Observation loop	SOCHGRID — entire architecture
10. Hallucination	Causes, types, all mitigation strategies	BOLVEDA <5% hallucination metric
Cheat Sheet	All definitions + top 10 Qs + your metrics	Read this 30 min before interview

Large Language Models (LLMs)

What they are, how they think, and how to explain them in 30 seconds.

One-line definition: An LLM is a neural network trained on massive amounts of text to **predict the next word**. Through that single task at enormous scale, it learns language, knowledge, and reasoning.

The Core Intuition

Imagine you have read every book, every website, every research paper ever written. Someone gives you a sentence and asks 'what word comes next?' If you have read enough, you answer accurately — not by memorising sentences, but by genuinely understanding language patterns, facts, and logic. That is exactly what an LLM does.

Example — Next-word prediction — the only thing an LLM does

Input: 'The capital of France is ____'

Output: Paris=94% Lyon=2% Berlin=0.1% ...

The model picks 'Paris'. This same mechanism at 1 trillion parameters = GPT-4.

The model does not 'know' Paris — it learned that 'Paris' follows that phrase with very high probability across millions of documents.

Key Properties

Probabilistic	Output is a probability distribution — not a lookup table. Same input can produce different outputs.
Autoregressive	Generates one token at a time. Each generated token feeds back as input for the next.
Stateless	NO memory between conversations. Knows only (a) training weights and (b) the current prompt.
Emergent	Coding, translation, and reasoning were NOT programmed — they emerged from scale.
Generalist	One model handles translation, coding, summarisation, QA — no task-specific design needed.

Base Model vs Instruction-Tuned Model

This is asked in almost every LLM interview. Know it cold.

Property	Base Model	Instruction-Tuned
Training	Next-token prediction on raw internet text	SFT + RLHF on (prompt, ideal response) pairs

Behaviour	Continues text. No instruction-following.	Follows instructions helpfully and safely.
Examples	LLaMA base weights, GPT-3 raw	ChatGPT, Claude, Gemini, GPT-4o
Hallucination	Very high	Lower (not zero)
In your projects	Starting point for fine-tuning	What you call via OpenAI API in BOLVEDA

Why this matters for BOLVEDA

GPT-4o is instruction-tuned. This is WHY it reliably follows your anti-hallucination system prompt.

A base model would ignore your 'Answer ONLY from context' instruction and hallucinate freely.

RLHF trained it to follow instructions — that is what makes your RAG pipeline work.

Temperature — How the Model Picks the Next Token

After computing probabilities for all possible next tokens, the model must pick one. Temperature controls how it picks.

Temperature = 0	Always pick highest-probability token. Deterministic. Same input always gives same output. Use for BOLVEDA QA.
Temperature = 1	Sample from probabilities as-is. Balanced. Default for most chat applications.
Temperature > 1	Flatten distribution — unlikely tokens get more chance. Creative but higher hallucination risk.
Top-p (nucleus)	Sample from smallest set of tokens whose cumulative probability reaches p (e.g. 0.9). Cuts off very unlikely tokens.

Parameters — What They Are

A parameter is a single floating-point number inside the model. GPT-4 has approximately 1 trillion parameters. Training sets every parameter to minimise prediction error. More parameters = more capacity for knowledge and complex patterns, but also more GPU memory, higher cost, and higher latency.

Interview Questions — This Topic

Q: What is an LLM and how does it actually work?

A: An LLM is a neural network trained on trillions of text tokens to predict the next token. Through this it learns language, facts, and reasoning. At inference it generates text autoregressively — one token at a time — sampling from a probability distribution over the vocabulary. Example: 'The capital of France is...' → model predicts 'Paris' with 94% probability. This same mechanism at scale produces GPT-4.

Tip: Always use the 'capital of France' example. Concrete and memorable.

Q: What is the difference between a base model and an instruction-tuned model?

A: A base model predicts next tokens from raw text — no concept of instructions. An instruction-tuned model has gone through SFT on human-written examples and RLHF to follow instructions helpfully and safely. ChatGPT and Claude are instruction-tuned. In BOLVEDA I call GPT-4o via API — that is instruction-tuned, which is why it reliably follows my anti-hallucination system prompt.

Tip: Connecting to your project shows practical understanding, not just textbook knowledge.

Q: What does temperature do and when would you set it to 0?

A: Temperature controls how the model samples from its probability distribution. At 0 it always picks the highest-probability token — deterministic and consistent. I set temperature=0 in BOLVEDA's QA step because I need consistent, reliable factual answers from documents, not creative variation.

Q: Why do LLMs sometimes give different answers to the same question?

A: Because LLMs sample from a probability distribution, not a lookup table. Unless temperature=0, there is inherent randomness in the sampling process. Even at temperature=0, slightly rephrased questions shift the probability distribution — so paraphrased inputs can yield different responses.

Tokenization

How text becomes numbers — and why this creates real engineering constraints.

One-line definition: Tokenization converts raw text into integer IDs (tokens) that the model processes. A token is a sub-word unit — not a character, not a word — produced by Byte-Pair Encoding (BPE).

Why Not Characters or Words?

Characters	Too granular. 'Hello' = 5 inputs. Sequences explode in length. Hard to learn word-level patterns.
Words	Vocabulary explodes (100K+ across all languages). Rare words are unseen. 'unhappiness' only works if it was in training vocab.
Tokens (BPE)	Sweet spot. ~50K vocabulary covers all common English text. 'unhappiness' → ['un','happiness']. Handles rare words gracefully.

Example — How GPT-4 tokenises text

"Hello, world!" → ["Hello", ",", " world", "!"] → [9906, 11, 1917, 0]

"unhappiness" → ["un", "happiness"] → [653, 66200]

"ChatGPT" → ["Chat", "G", "PT"] → [14304, 38, 2898]

Common words = 1 token. Rare/long words = multiple tokens. Each digit of a number can be a separate token.

Critical Facts for Interviews and Projects

- **1 token ≈ 0.75 English words** (≈ 4 characters). A 1,000-word essay ≈ 1,333 tokens.
- **APIs charge per token.** GPT-4o: ~\$5/million input tokens. A RAG query sending 2,500 tokens costs ~\$0.0125.
- **Context window is in tokens.** GPT-4o: 128K tokens. Your prompt + retrieved chunks + response all count toward this.
- **Non-English text uses 2–4× more tokens.** Hindi, Chinese, Arabic have fewer BPE merges — more tokens per word.
- **Numbers tokenise badly.** '12345' → ['1','2','3','4','5'] — 5 separate tokens. This is why LLMs fail at arithmetic.
- **Code is efficient.** 'def', 'return', 'import' are usually single tokens from heavy code training.

BOLVEDA token cost — real engineering thinking

512 tokens × 5 chunks retrieved = 2,560 tokens of context per query

+ system prompt (~200) + user question (~50) + answer (~300) = ~3,110 tokens per call

At \$5/M tokens → ~\$0.016 per query. 1,000 queries/day → ~\$16/day

Reducing from top-5 to top-3 chunk retrieval saves 40% of retrieval cost.

This is the kind of production-aware thinking that separates good candidates from great ones.

Interview Questions — This Topic

Q: What is a token and why does it matter for LLM applications?

A: A token is the basic text unit — roughly 0.75 English words or 4 characters. It matters for three reasons: cost (APIs charge per token), context limits (GPT-4o has 128K tokens shared between input and output), and quality (rare words split into many tokens reduce accuracy). In BOLVEDA each query sends about 3,000 tokens so I monitored and optimised token usage.

Tip: Mention the real cost calculation — shows you built something production-aware.

Q: Why are LLMs bad at arithmetic?

A: Numbers tokenise poorly — '12345' becomes ['1','2','3','4','5']. The model reasons over tokens as patterns, not as numerical values. It pattern-matches arithmetic from training examples rather than actually computing. The fix is tool use — give the model a calculator or code execution environment.

The Transformer & Self-Attention

The architecture behind every modern LLM — conceptual understanding is enough for interviews.

One-line definition: The Transformer is a neural network architecture that uses **self-attention** to process all tokens simultaneously and compute relationships between every pair of tokens. Introduced in 2017, it replaced RNNs and powers every modern LLM.

Why Transformers Beat the Old Approach (RNNs)

RNN (old)	Processed text left-to-right, one token at a time. By token 500, information from token 1 was diluted. Could not parallelise training.
Transformer	Processes all tokens simultaneously via matrix operations. Self-attention lets any token attend to any other token directly. Massively parallelisable on GPUs.

Self-Attention — The Core Innovation

Self-attention answers one question: **For each token, which other tokens are most important for understanding it?** It computes a score between every pair of tokens and uses those scores to build a weighted representation.

Example — Self-attention resolves pronouns — impossible for old RNNs

"The trophy did not fit in the suitcase because it was too big."

When processing 'it', self-attention assigns high weight to 'trophy' (not 'suitcase').

The model correctly resolves that 'it' refers to 'trophy' by attending to all tokens at once.

RNNs failed at this for long sequences because 'trophy' was many sequential steps back.

Query, Key, Value (Q, K, V) — Interview Mechanics

Think of self-attention like a search engine inside the model. Each token is projected into three vectors:

- **Query (Q):** What am I looking for? The current token asking 'who is relevant to understanding me?'
- **Key (K):** What do I offer? Every other token advertising what it represents.
- **Value (V):** What information do I contain? The actual content to pass forward.

$$\text{Attention score} = \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V$$

Division by $\sqrt{d_k}$ prevents dot products from becoming too large, causing vanishing gradients in softmax.

Multi-Head Attention — Why Multiple Heads?

Instead of one attention operation, run several in parallel ('heads'). Each head can specialise in different relationship types: syntax, coreference, positional proximity. All head outputs are concatenated and projected. GPT-2: 12 heads. GPT-3: 96 heads.

Full Transformer Block (Stacked N Times)

Token Embeddings	Convert token IDs to dense vectors (768–12,288 dimensions depending on model size).
Positional Encoding	Add position info — Transformer processes all tokens in parallel, has no built-in order.
Multi-Head Attention	Each token attends to all others. Finds relationships across the whole sequence.
Feed-Forward Layer	Two linear layers + GELU activation per token. Stores most of the model's factual knowledge.
Layer Normalisation	Normalises activations for stable training. Applied before each sub-layer in modern models.
Residual Connection	Output = Input + Sublayer(Input). Lets gradients flow through 100+ layer deep networks.

Watch Out / Common Mistake

Do NOT say 'Transformer is just attention.' The feed-forward layers store most of the model's factual knowledge and account for 2/3 of parameters. Attention finds relationships; FFN stores facts.

Interview Questions — This Topic

Q: Explain self-attention in simple terms.

A: Self-attention lets each token look at all other tokens and figure out which are most relevant for understanding it. For each token we compute a query (what am I looking for) and compare it against every other token's key (what do they offer). The scores determine how much each token's value (information) contributes to the current token's representation. Classic example: 'The trophy didn't fit in the suitcase because it was too big' — when processing 'it', attention correctly assigns high weight to 'trophy' over 'suitcase'.

Tip: Use the pronoun resolution example — it's concrete, memorable, and shows depth.

Q: Why does the Transformer use positional encoding?

A: The Transformer processes all tokens in parallel via matrix operations — unlike RNNs it has no built-in sense of order. Without positional encoding, 'dog bites man' and 'man bites dog' produce identical representations. Positional encodings add position information to token embeddings so the model learns order-dependent patterns.

Q: What is multi-head attention and why multiple heads?

A: Multi-head attention runs several attention operations in parallel, each with different learned projection matrices. Each head can specialise in different relationship types — syntax, coreference, positional proximity. Concatenating all heads gives richer representations than a single operation. GPT-2 has 12 heads, GPT-3 has 96.

Training LLMs & RLHF

Pre-training, fine-tuning, and how raw models become helpful assistants.

One-line definition: Pre-training teaches the model language and knowledge through next-token prediction on trillions of tokens. RLHF then makes it helpful, harmless, and honest by training on human preferences.

Stage 1: Pre-Training

The model sees trillions of tokens from web pages, books, code, Wikipedia, and papers. For each position, it predicts the next token. Loss = cross-entropy between predicted probabilities and actual next token. Backpropagation + AdamW optimiser updates all parameters. Repeated billions of times across thousands of GPUs for weeks.

Example — The compression insight — why pre-training actually works

To predict 'The mitochondria is the _____ of the cell' correctly, the model must have learned biology.

To predict legal text correctly, it must have learned legal reasoning.

Compressing all of human text into accurate predictions REQUIRES genuine understanding.

Pre-training cost: GPT-4 ~\$50–100M. LLaMA 3 70B: ~6,000 A100s × several weeks.

Stage 2: Supervised Fine-Tuning (SFT)

Human contractors write 10,000–100,000 high-quality (prompt, ideal response) pairs. The model is fine-tuned on these to learn what helpful, safe responses look like. After SFT the model follows instructions but may not reliably match human preferences.

Stage 3: RLHF — Reinforcement Learning from Human Feedback

Step A: Preferences	SFT model generates multiple responses per prompt. Humans rank best to worst. Creates a preference dataset.
Step B: Reward Model	Separate neural network trained to predict human preference scores. Learns what 'good' looks like.
Step C: PPO	The SFT model (policy) generates responses, reward model scores them, PPO updates model to score higher. KL-penalty prevents reward hacking.

KL-divergence penalty keeps the model from drifting too far from SFT — prevents it from generating gibberish that tricks the reward model (reward hacking).

DPO	Direct Preference Optimisation — skips reward model, fine-tunes directly on preference pairs. Simpler, increasingly popular.
------------	--

RLAIF	RL from AI Feedback — uses Claude or GPT-4 instead of humans to rank responses. Scalable.
Constitutional AI	Anthropic's method. Model critiques its own outputs against a 'constitution' of principles and revises.

Why RLHF matters for BOLVEDA

GPT-4o went through SFT + RLHF. This is WHY it follows your anti-hallucination system prompt reliably.

A raw base model would ignore 'Answer ONLY from context' and hallucinate freely.

RLHF trained it to follow instructions — that's what makes your RAG pipeline work end-to-end.

Interview Questions — This Topic

Q: What is RLHF and why is it necessary?

A: RLHF = Reinforcement Learning from Human Feedback. Pre-training gives language ability but not alignment. RLHF: (1) collect human preference rankings of model outputs, (2) train a reward model to predict them, (3) use PPO to optimise the LLM to score higher. This aligns the model to be helpful, harmless, and honest. Three stages: SFT → reward model → PPO.

Q: What is the difference between pre-training and fine-tuning?

A: Pre-training trains from scratch on trillions of tokens from random initialisation — sets all parameters. Fine-tuning continues training a pre-trained model on a smaller task-specific dataset. Pre-training costs millions of dollars and weeks. Fine-tuning is cheap (hours to days). Almost all developers fine-tune existing models. LoRA makes it cheaper still — only small adapter matrices are trained.

Embeddings

The mathematical backbone of BOLVEDA Layer 3 — the most important NLP concept to know deeply.

One-line definition: An embedding is a dense vector of floating-point numbers representing the **meaning** of text in high-dimensional space. Similar meanings produce geometrically similar vectors. This is what makes semantic search possible.

The Intuition — Meaning as a Point in Space

Imagine a 1,536-dimensional space (OpenAI's embedding size). Every piece of text maps to exactly one point in this space. Text with similar meaning maps to nearby points. Text with completely different meaning maps to distant points. We measure 'nearness' using cosine similarity.

Example — What vectors actually capture

"What is RAG?" and "Explain retrieval augmented generation" → cosine similarity ≈ 0.95

"What is RAG?" and "How do I make pasta?" → cosine similarity ≈ 0.1

"King" - "Man" + "Woman" ≈ vector for "Queen" (the famous Word2Vec arithmetic result)

Embeddings capture semantic relationships — not just word overlap. This beats keyword search.

Keyword Search vs Semantic Search

Property	Keyword Search (BM25)	Semantic / Embedding Search
Mechanism	Count matching words	Compare meaning vectors
Query: 'fix a car'	Needs 'fix' and 'car' in document	Finds 'automobile repair' too
Synonyms	Misses ('vehicle' ≠ 'car')	Handles naturally
In BOLVEDA	Not used	ChromaDB cosine similarity

Cosine Similarity — The Distance Metric

Cosine similarity measures the **angle** between two vectors, not their magnitude. Range: -1 (opposite meaning) to +1 (identical meaning). In practice, similarity above 0.8 usually means closely related content.

$$\text{cosine_similarity}(A, B) = (A \cdot B) / (|A| \times |B|)$$

Example — Why cosine — not Euclidean distance?

"I love dogs" vs "I love dogs. I love dogs. I love dogs." (text repeated 3×)

Euclidean distance: very different (vectors have different magnitudes/lengths).

Cosine similarity: 1.0 (same direction = same meaning).

For semantic search we care about DIRECTION (what the text means), not LENGTH (how much text).

Key Embedding Models to Know

text-embedding-ada-002	OpenAI. 1,536 dimensions. \$0.02/1M tokens. Used in BOLVEDA Layer 3.
text-embedding-3-small	OpenAI. 1,536 dims. Better quality than ada-002. Same cost.
all-MiniLM-L6-v2	Open source (HuggingFace). 384 dims. Free. Great for prototypes.
BGE-large	Open source. Often beats OpenAI on benchmarks. Free. Best if you want zero API cost.
Gemini embedding-001	Google. 768–1,536 dims. Also mentioned in BOLVEDA architecture.

BOLVEDA Layer 3 — your code in plain English

Step 1: Take a 512-token text chunk from a PDF page.

Step 2: POST to text-embedding-ada-002 API.

Step 3: Receive a Python list of 1,536 floats. That list IS the embedding.

Step 4: Store [1,536 floats + original text + metadata] in ChromaDB.

Step 5: At query time — embed the user's question (another 1,536 floats).

Step 6: ChromaDB computes cosine similarity → returns top-5 most similar chunks.

Interview Questions — This Topic

Q: What is an embedding and why is it useful?

A: An embedding is a dense vector of floating-point numbers representing the meaning of text in high-dimensional space. Text with similar meaning has similar vectors, enabling semantic search — finding relevant documents by meaning rather than keyword overlap. In BOLVEDA I embed every PDF chunk and every user question using text-embedding-ada-002 (1,536 dimensions), then find closest chunks via cosine similarity in ChromaDB.

Tip: Mention the specific dimension — 1,536 for text-embedding-ada-002. Shows you actually implemented it.

Q: Why cosine similarity instead of Euclidean distance?

A: Cosine similarity measures the angle between vectors, not their length. The same text repeated twice has double the magnitude but identical meaning — Euclidean distance says they're far apart, cosine correctly returns 1.0. For semantic search we care about what the text means (direction), not how long it is (magnitude).

Q: What is the difference between a generative model and an embedding model?

A: A generative model like GPT-4 produces text tokens autoregressively — you give a prompt, it generates a response. An embedding model like text-embedding-ada-002 takes text and produces a single fixed-size vector representing its meaning. In BOLVEDA I use both: the embedding model for retrieval, and GPT-4o for generating the final answer.

Tip: This shows you understand the full RAG architecture, not just individual pieces.

RAG — Retrieval-Augmented Generation

The complete 6-stage pipeline behind BOLVEDA. Practice drawing this on a whiteboard.

One-line definition: RAG gives an LLM access to an external knowledge base at query time. Instead of relying on memorised training data, the model retrieves relevant documents and uses them as context to generate grounded, cited, accurate answers.

The Three Problems RAG Solves

Knowledge cutoff	LLMs have a training cutoff and don't know recent events. RAG retrieves current documents at inference time.
Private documents	Your PDFs, company docs, contracts were never in training data. RAG gives the model access to them.
Hallucination	Without RAG, model guesses from memory. With RAG, the answer is in the prompt — model reads, not invents.

The Complete 6-Stage BOLVEDA Pipeline

Stage 1: Document Ingestion

Load PDF with pdfplumber or PyPDF2. Extract raw text page by page. Clean headers, footers, page numbers, and extra whitespace. Output: a long clean string of text per document.

Stage 2: Chunking — The Most Critical Decision

Split the long text into smaller chunks. Each chunk must fit in a context window slot and contain enough context to be meaningful on its own. Wrong chunk size destroys retrieval quality.

Example — Why 512 tokens with 50-token overlap (BOLVEDA's choice)

Too small (100 tokens): chunks lack context. An answer spanning two chunks gets missed.

Too large (2,048 tokens): one chunk covers many topics — retrieval is imprecise.

512 tokens: well-tested sweet spot. One coherent idea fits in 512 tokens.

50-token overlap: if an answer falls near a chunk boundary, it appears in at least one chunk.

You tried 200-token chunks first — they cut sentences mid-way. 512 fixed it. TELL THIS STORY.

Stage 3: Embedding Each Chunk

Send each chunk to text-embedding-ada-002. Receive a 1,536-dimensional vector — the 'fingerprint' of that chunk's meaning. Store (vector + original text + metadata) in ChromaDB.

Stage 4: Vector Store — Build the Index

ChromaDB builds an HNSW index over all stored vectors. This enables millisecond cosine similarity search across thousands of chunks without comparing against every single vector.

Stage 5: Query → Retrieve → Augment

User speaks. Whisper API transcribes speech to text. The question is embedded (same model, same 1,536 dims). ChromaDB finds top-5 most similar chunks via cosine similarity. Those 5 chunks are injected into the GPT-4o prompt as context.

Stage 6: LLM Generation with Anti-Hallucination Prompt

Prompt = [system: anti-hallucination instructions] + [context: 5 retrieved chunks] + [user: question]. GPT-4o reads all of this and generates a grounded, cited answer. ElevenLabs converts answer to speech.

BOLVEDA Anti-Hallucination System Prompt:

"Answer ONLY from the provided context below."

"If the answer is not in the context, say: I don't know based on the provided documents."

"Always cite which section or page you used."

"Do not use any knowledge from your training data."

Why this achieves <5% hallucination: the ground truth is literally in the prompt. Model reads, not invents.

RAG vs Fine-Tuning — Decision Framework

Use RAG when...	Use Fine-Tuning when...
Knowledge changes frequently (live docs, news)	Teaching a new task style or output format
You need citations and source attribution	Adapting tone or style consistently
Private documents not in any training data	Proprietary domain vocabulary model lacks
Low budget — no retraining needed	You have 1,000+ high-quality labelled examples
Quick prototype (days, not weeks)	Need lower latency (smaller tuned model)

Advanced RAG Techniques (Bonus Points)

Hybrid Search	Combine BM25 (keyword) + embedding (semantic) scores. Best of both worlds.
Reranking	After top-K retrieval, a cross-encoder reranks results for better precision.
HyDE	Generate a hypothetical answer, embed it, search for similar chunks. Improves recall.
Multi-query	Rephrase user query 3 ways, retrieve for each, deduplicate. Improves coverage.
Parent-child chunks	Retrieve small chunks for precision, fetch their larger parent for context.

Interview Questions — This Topic

Q: Walk me through BOLVEDA's RAG pipeline end to end.

A: Six stages: (1) PDF ingestion with pdfplumber — clean text extraction. (2) RecursiveCharacterTextSplitter: 512-token chunks with 50-token overlap. (3) Each chunk embedded via text-embedding-ada-002 → 1,536-dim vector. (4) ChromaDB stores vectors + text with HNSW index. (5) User question transcribed by Whisper, embedded, top-5 cosine-similar chunks retrieved. (6) GPT-4o generates grounded answer using anti-hallucination prompt. ElevenLabs speaks it.

Tip: Practice this in under 90 seconds. This is your primary project explanation.

Q: Why RAG over fine-tuning for BOLVEDA?

A: Three reasons. First, users upload their own private PDFs — not in any model's training data, so fine-tuning wouldn't help. Second, RAG enables source citations so users know exactly which page the answer came from. Third, RAG took days to build and test. Fine-tuning would take weeks and significant compute cost.

Q: Why not just paste the whole PDF into the context window?

A: A 200-page textbook is ~150K tokens — won't fit in GPT-4o's 128K context. Even if it fit, longer context means proportionally higher cost and latency. RAG retrieves only the 5 most relevant chunks (~2,500 tokens). Also: the 'Lost in the Middle' problem — models pay less attention to information buried deep in long prompts. Selective retrieval is better than full-document stuffing.

Q: Why 512 tokens with 50-token overlap?

A: 512 is a well-tested sweet spot — enough context for one coherent idea, small enough for precise retrieval. The 50-token overlap prevents losing information at chunk boundaries. I first tried 200-token chunks — they cut sentences mid-way. Switching to 512 with 50-token overlap fixed it. I always mention this iteration story in interviews.

Vector Databases

ChromaDB in BOLVEDA — what they are, how they work, why regular SQL won't do.

One-line definition: A vector database stores high-dimensional embedding vectors and provides fast **Approximate Nearest Neighbour (ANN) search** — finding the most similar vectors to a query in milliseconds, even across millions of entries.

Why Not a Regular Database?

SQL exact match	WHERE vector = query_vector doesn't work. You can't 'equal' a 1,536-float array.
SQL brute force	You could compute cosine similarity for every row. With 1M chunks = 1M dot products per query. Way too slow.
Vector DB	Builds a specialised index (HNSW). Navigates to nearest neighbours in $O(\log n)$. Millisecond retrieval at scale.

HNSW — The Index Algorithm ChromaDB Uses

Hierarchical Navigable Small World (HNSW) is a graph-based ANN index. It builds a multi-layer graph where upper layers are coarse (few nodes, long edges) and lower layers are dense (all nodes, short edges). Search starts at top and drills down. Result: $O(\log n)$ search — milliseconds instead of seconds.

You do NOT need to understand HNSW deeply. Just say: 'ChromaDB uses an HNSW index for fast ANN search, giving millisecond retrieval even on large collections.'

ChromaDB — BOLVEDA's Choice

Type	Open-source. Runs entirely locally. No API key. No cloud. No data egress.
Storage	Stores vectors + original text + metadata in local SQLite by default. Persistent across sessions.
Key API	<code>collection.add(embeddings=[...], documents=[...], ids=[...])</code> and <code>collection.query(query_embeddings=[...], n_results=5)</code>
Why local	User PDFs stay on their machine. No privacy concerns. No network latency to cloud.
Limitation	Not designed for millions of vectors in production. Use Pinecone or Qdrant for that.

Vector Database Comparison

Database	Type	Best For	Key Note
----------	------	----------	----------

ChromaDB	Local/Open	Prototypes, local apps	Used in BOLVEDA. No cloud required.
FAISS	Library	Research, in-memory	Meta's library. No persistence.
Pinecone	Cloud SaaS	Production at scale	Managed, free tier available.
Weaviate	Open/Cloud	Hybrid BM25 + vector	Best of both search types.
pgvector	PostgreSQL ext	Existing Postgres users	Add vectors to SQL DB.
Qdrant	Open/Cloud	High performance	Rust-based, very fast.

Interview Questions — This Topic

Q: What is a vector database and why can't you use SQL?

A: A vector database stores high-dimensional embeddings and provides fast ANN search. Regular SQL does exact matching — it can't efficiently find the 'closest' vector across millions of rows. Vector databases use HNSW index structures to do this in milliseconds. In BOLVEDA I use ChromaDB because it runs locally — user PDFs never leave their machine.

Q: Why ChromaDB over Pinecone for BOLVEDA?

A: ChromaDB runs locally — user PDFs stay on their device, no data egress, no privacy concerns. It persists data by default, has a clean Python API, and takes under 10 minutes to set up. For production at millions of documents I'd migrate to Pinecone or Qdrant. Right tool for the right stage.

Tip: Always show trade-off awareness — prototype choice vs production choice.

Prompt Engineering

The most practical skill — writing prompts that get reliable, high-quality outputs every time.

One-line definition: Prompt engineering is the practice of designing LLM inputs to **reliably** produce high-quality outputs. No model training required. Works immediately. The most impactful skill for building LLM applications.

Core Techniques

1. Zero-Shot Prompting

Give the model a task with no examples. Works for clear, simple tasks.

```
Classify this review as Positive, Negative, or Neutral: "The product arrived late but the quality was excellent."
```

2. Few-Shot Prompting

Provide 2–5 input/output examples before the task. The model infers the pattern.

```
Review: "Arrived broken." → Negative
```

```
Review: "Fast delivery, great!" → Positive
```

```
Review: "Terrible, never again." → ???
```

The model answers 'Negative' from the pattern — no explicit instruction needed.

3. Chain-of-Thought (CoT) Prompting

Tell the model to think step by step. Dramatically improves multi-step reasoning, math, and logic.

```
Without CoT: 'What is  $17 \times 24$ ?' → Model may say 418 (wrong)
```

```
With CoT: 'What is  $17 \times 24$ ? Think step by step.'
```

```
→ ' $17 \times 20 = 340$ ,  $17 \times 4 = 68$ ,  $340 + 68 = 408$ ' ✓
```

Just adding 'Let's think step by step.' to any complex task significantly improves accuracy.

4. System Prompt / Role Prompting

Define the model's persona and rules in the system prompt. All responses follow these rules.

```
System: "You are BOLVEDA, a document assistant. Answer ONLY from the provided context. If not in context, say: I don't know based on the provided documents. Always cite which section or page you used."
```

Best Practices

- **Be specific about format:** 'Respond in JSON with keys: answer, confidence, source_page' beats 'give me the answer.'
- **Use delimiters:** Separate context from instructions with triple-backticks or XML tags.

- **Positive beats negative:** 'Answer only from context' works better than 'don't make things up.'
- **Critical instructions at top AND bottom:** 'Lost in the Middle' — models pay less attention to instructions buried in long prompts.
- **Test adversarially:** Try questions NOT in your documents. Verify the fallback actually works.

Advanced Techniques

ReAct	Reason + Act: interleave Thought (reasoning) with Action (tool call). Used in SOCHGRID.
Self-consistency	Generate 5 responses, take majority. Filters random hallucinations.
Self-refine	Generate answer, critique it, revise. Loop 2–3 times. Improves quality on complex tasks.
Structured output	Instruct model to respond in JSON/Pydantic schema. Used in SOCHGRID for parseable reports.

Interview Questions — This Topic

Q: What is chain-of-thought prompting and when would you use it?

A: CoT prompting tells the model to reason step by step before giving a final answer. It dramatically improves performance on multi-step reasoning, math, and logic tasks. Trigger it with 'think step by step' or by showing examples with intermediate steps. Use for complex tasks — not needed for simple classification.

Q: How did you design the anti-hallucination prompt in BOLVEDA?

A: Three elements: hard constraint ('Answer ONLY from context'), explicit fallback ('say I don't know if not in context'), enforced attribution ('always cite the section'). I tested this with a 30-question test set including out-of-scope questions — verified the model correctly says it doesn't know. Result: <5% hallucination rate, 87% answer accuracy.

Tip: The 30-question test set is the detail that turns this from a demo into an engineered system.

Agents, Tool Use & ReAct

SOCHGRID's core — autonomous AI that thinks, searches, and produces structured reports.

One-line definition: An LLM agent is a system where the model is given tools (functions it can call) and **autonomously decides** which tools to use, when, and with what arguments to complete a goal. The model plans and acts — it doesn't just respond.

Regular LLM Call vs Agent

Regular LLM call	You send a prompt. Model responds once. Done. One round trip. You control everything.
Agent	Model decides to call tools. Tool results feed back in. Model reasons again. Multiple iterations. Model is the decision-maker.
The key shift	In an agent, the LLM is the controller — it decides what to do next, not the developer.

The ReAct Pattern — Reason + Act

The model alternates between Thought (internal reasoning), Action (calling a tool), and Observation (reading the tool output). This loop repeats until the model decides it has enough information.

SOCHGRID ReAct loop — researching 'EV adoption in India 2025'

Thought: I need EV adoption statistics for India. I'll search for this.

Action: web_search('EV adoption India 2025 statistics')

Observation: [5 search results with URLs and snippets]

Thought: I have adoption data but missing charging infrastructure info.

Action: web_search('EV charging stations India 2025 count')

Observation: [5 more results]

Thought: I now have enough. Generating report.

Action: finish_tool({'title': '...', 'key_findings': [...], 'sources': [...]}))

The LLM makes every decision. You (developer) just provide the tools and max_iterations cap.

SOCHGRID Architecture — All 5 Steps

Step 1: Input	Research topic entered via Streamlit. ONLY human input. Everything after is autonomous.
Step 2: Agent Init	LangChain: ZERO_SHOT_REACT_DESCRIPTION + GPT-4o + [search_tool, finish_tool] + max_iterations=8. ZERO_SHOT = no examples needed — model figures out when to use each tool from descriptions.

Step 3: Search Loop	Tavily Search API (free: 1,000/month). requests + BeautifulSoup scrapes full content of top 2 URLs. After each search: do I have enough? If no → search again. Up to 8 iterations.
Step 4: Pydantic Output	Agent fills a strict schema: ResearchReport(title, summary, key_findings: List[str], pros, cons, sources). Always parseable. Always consistent. Unit-testable.
Step 5: Display	Streamlit renders report + source links. Export Markdown or PDF. Total: under 30 seconds.

Engineering Decisions That Impress Interviewers

max_iterations=8	Without this, agent loops forever on complex topics. After 8 searches, forces report generation regardless.
Source quality filter	Blocks Reddit, Quora, ad-heavy sites. Only .edu, .org, reputable news = better reports.
Hallucination control	'Only use facts from your search observations. Never add from training memory.' Each claim traces to a search result.
Pydantic schema	Enforces consistent output format. Eliminates parsing failures. Makes output unit-testable.

Agent Frameworks to Know

LangChain	Most popular. General-purpose agents + RAG chains. Used in both BOLVEDA and SOCHGRID.
LlamaIndex	Better for complex RAG pipelines. More advanced indexing features.
CrewAI	Multi-agent collaboration. Define agents with roles (researcher, writer, reviewer).
AutoGen	Microsoft's framework. Agents critique each other. Good for complex reasoning.
Pydantic AI	Type-safe structured output. Excellent for SOCHGRID-style structured reports.

Interview Questions — This Topic

Q: What is an AI agent and how is it different from a regular LLM call?

A: An LLM agent is given tools and autonomously decides which to use, when, and how many times to complete a task. A regular call is one round trip — prompt in, response out. An agent loops: model reasons, calls a tool, reads result, reasons again, repeats until done. In SOCHGRID the model autonomously does 3–8 searches before generating the final report — I don't control that loop at all.

Q: Explain the ReAct pattern.

A: ReAct = Reason + Act. The model alternates Thought → Action → Observation in a loop until it produces a final answer. In SOCHGRID: 'I need EV data → search → got adoption data, need infrastructure data → search again → now I have enough → generate report.' The LLM makes every decision in this loop.

Tip: Give the SOCHGRID example — makes it concrete and memorable.

Q: How did you prevent SOCHGRID from running forever?

A: I set max_iterations=8 in LangChain's agent initialisation. After 8 searches, the agent is forced to generate the report regardless. Without this cap, on complex topics the model keeps deciding it needs more data and never stops. Most SOCHGRID reports complete in 3–5 iterations — the cap is a safety net, not a constraint.

Q: Why Pydantic for SOCHGRID's output instead of free-text?

A: Free-text report generation is inconsistent — different formats, sometimes missing sections. Pydantic enforces a strict schema: title, summary, key_findings as List[str], pros, cons, sources. Benefits: output is always parseable, Streamlit reliably renders each field, I can write unit tests to validate structure. This shows production thinking — not just 'it works' but 'it works consistently and is testable.'

Hallucination — Causes & Mitigations

BOLVEDA's central engineering challenge and your key differentiating metric.

One-line definition: Hallucination is when an LLM generates text that is confident, fluent, and factually wrong or completely fabricated. The model doesn't know it is wrong — it outputs the most probable-sounding continuation regardless of truth.

Why Does Hallucination Happen?

Training objective	Trained to minimise token prediction error — not to only assert verified facts. Plausible-sounding text has low loss whether true or false.
No knowledge DB	Weights store compressed patterns, not a structured queryable database. Facts are baked in approximately.
No uncertainty output	The model doesn't output a confidence score. Generates tokens regardless of certainty.
Rare fact extrapolation	Facts rare in training data are poorly learned. Model fills gaps by pattern-matching similar-sounding facts.
Long context drift	In very long conversations, model can contradict earlier constraints or lose track of instructions.

Watch Out / Common Mistake

"What papers has Prof. X published?" → Model invents plausible-sounding paper titles that don't exist.

"What legal precedent covers X?" → Model cites a case with a real-sounding name never decided.

"What did CEO X say about Y?" → Model invents a quote in the CEO's style.

Danger: hallucinated output is indistinguishable from correct output without verification.

Mitigation Strategies — Ranked by Effectiveness

RAG ★★★★★	Best fix. Provide the answer in the prompt. Model reads documents, not memory. BOLVEDA: <5% hallucination.
Anti-hallucination prompt ★★★★★	'Answer ONLY from context. If not in context, say I don't know.' Forces grounding.
Temperature = 0 ★★★	Deterministic output. Removes sampling randomness producing unexpected tokens.
Source citation enforcement ★★★	Force model to cite every claim. Uncited claims become visible and verifiable.

Self-consistency ★★★	Generate 5 responses, take majority answer. Random hallucinations disagree; correct answers converge.
Tool use ★★★★★	Calculator, search engine, code execution. Replace memory-reliant tasks with verified outputs.
Verification prompt ★★	After generating: 'List any claims you are uncertain about.' Surfaces potential issues.

Interview Questions — This Topic

Q: What is hallucination and why does it happen?

A: Hallucination is when an LLM generates confident but factually wrong content. It happens because the model is trained to produce plausible text, not verified facts. It has no structured knowledge store — just compressed patterns. For rare or unknown facts, it extrapolates from similar patterns and produces something plausible-sounding but false. The model has no way to express uncertainty.

Q: How did you achieve under 5% hallucination in BOLVEDA?

A: Two things working together: RAG (inject actual PDF text into the prompt — model has the answer and doesn't need memory) + anti-hallucination prompt ('Answer ONLY from context. Say I don't know if not in context'). I validated this with a 30-question test set covering factual, inferential, and out-of-scope questions. Result: 87% answer accuracy, under 5% hallucination rate.

Tip: The 30-question test set is what transforms this from a weekend demo into an engineered system. Always mention it.

Q: What is the difference between hallucination and a factual error?

A: Hallucination specifically means confident fabrication — the model generates content that sounds credible but has no basis in fact or provided context, often for things it simply doesn't know. A factual error is broader: outdated info, misunderstandings, logical mistakes. Hallucination is the most dangerous because the output is indistinguishable from correct output without independent verification.

MASTER CHEAT SHEET

Print this page. Read it 30 minutes before your interview.

30-Second Definitions

LLM	Neural network trained on text to predict next tokens. Learns language, knowledge, reasoning through compression.
Token	~0.75 English words or 4 characters. Basic unit of LLM I/O. APIs charge per token. GPT-4o: 128K context limit.
Temperature	Controls randomness. 0 = greedy/deterministic. Use 0 for factual tasks (BOLVEDA QA step).
Hallucination	Confident but wrong output. Fixed by: RAG + anti-hallucination prompt + temperature=0. BOLVEDA: <5%.
Embedding	Dense vector of floats representing text meaning. Similar meaning = similar vectors. 1,536 dims in BOLVEDA.
Cosine similarity	Angle between two vectors. 1.0 = identical meaning. 0 = unrelated. Used in ChromaDB retrieval.
RAG	Retrieve relevant docs → inject into prompt → LLM answers from context, not memory. Core of BOLVEDA.
Chunking	Split docs into 512-token pieces with 50-token overlap. Overlap prevents losing info at boundaries.
Vector DB	Stores embeddings for fast ANN search. BOLVEDA uses ChromaDB (local, no data egress, no privacy risk).
RLHF	SFT + human preference ranking + reward model + PPO. Turns base model into helpful assistant.
Agent	LLM + tools + decision loop. Model autonomously chooses what to call. SOCHGRID: search, reason, report.
ReAct	Thought → Action → Observation loop. Model reasons, calls tools, reads results, repeats until done.
Transformer	Attention-based neural architecture. Self-attention: every token relates to every other token.
Self-attention	Each token computes relevance to all other tokens simultaneously. Core Transformer innovation.
CoT	Chain-of-Thought: 'think step by step.' Dramatically improves multi-step reasoning tasks.
Pydantic	Python schema validation. SOCHGRID uses it to enforce consistent, parseable, testable agent output.

BPE	Byte-Pair Encoding. Algorithm that learns sub-word tokenisation by merging frequent pairs 50K+ times.
ANN	Approximate Nearest Neighbour. Fast vector search — tiny accuracy loss for massive speed gain.
HNSW	Graph-based ANN index used in ChromaDB. Multi-layer: coarse top, dense bottom. $O(\log n)$ search.
KV Cache	Cache of Key/Value attention computations. Speeds up sequential token generation at inference time.

The 10 Most Asked LLM Interview Questions

1	What is an LLM?	Next-token predictor on trillions of tokens. Learns language, knowledge, reasoning through compression.
2	What is RAG?	Retrieve relevant docs, inject into prompt, LLM answers from context not memory. Solves hallucination + private data.
3	Hallucination — what is it? How do you fix it?	Confident wrong output. Fix: RAG + anti-hallucination prompt + temperature=0 + source citations.
4	What is an embedding?	Dense vector representing text meaning. Similar text → similar vectors. Enables semantic search.
5	What is RLHF?	SFT → reward model from human preferences → PPO. Turns base model into helpful, aligned assistant.
6	RAG vs fine-tuning?	RAG: private/changing data, citations, fast build. Fine-tune: new task formats, consistent style.
7	What is a token?	~0.75 words. APIs charge per token. Context window measured in tokens. GPT-4o: 128K.
8	What is self-attention?	Each token computes relevance to all other tokens simultaneously. Core Transformer innovation.
9	What is an agent?	LLM + tools + ReAct loop. Model decides what to call. SOCHGRID: 3–8 searches per report autonomously.
1 0	What is temperature?	Controls output randomness. 0=greedy (deterministic, factual). 1=standard. Higher=creative/risky.

Your Project Numbers — Say These Without Looking

BOLVEDA	Value	SOCHGRID	Value
Answer accuracy	≥87%	Quality score (user ratings)	≥83%
Hallucination rate	<5%	Sources per report	6–8 average

End-to-end latency	<3 seconds	Time per report	<30 seconds
Whisper STT accuracy	≥94%	Test tasks validated	20+ across 5 domains
Test set size	30 Q&A; pairs	Max search iterations	8 per task
Chunk size	512 tokens + 50 overlap	Output format	Pydantic schema
Embedding model	text-embedding-ada-002	Search API	Tavily (1,000/mo free)

The single most important thing to remember

You don't just know LLM theory — you BUILT with it.

BOLVEDA and SOCHGRID are real, measured systems with real engineering decisions:

- Chunk size of 512 (not 200) — because you tested it and saw the difference
- 50-token overlap — because you saw what happened without it
- max_iterations=8 — because you saw the agent loop indefinitely
- Pydantic schema — because free-text output was unparseable across runs
- Anti-hallucination prompt tested with 30 adversarial questions

Every interview question about LLMs can be answered with:

'Here is exactly how I handled this in my project.'

That is the difference between a textbook candidate and a builder.